Namma Drishti — Technical Report

Offline edge object detection and counting for safer streets

Namma AI Hackathon 2026 · Problem Statement 2 · [Team name] · [College / organisation]

Abstract

Namma Drishti is a functional edge-AI proof of concept that detects and counts pedestrians and vehicles in a short street clip or a set of images, entirely on a Raspberry Pi. It uses an Arm-optimized YOLO11n INT8 ExecuTorch model (4.73 MB) running on the CPU through XNNPACK. The system writes per-frame counts, an annotated evidence bundle and a latency report; it makes no network calls and extracts no identities. On Arm's published Raspberry Pi 5 reference, the model runs at 134.10 ms p50 (7.46 FPS) with 59.17 MB peak memory and 38.90% mAP@0.5:0.95 on COCO val2017. On the committed 15-second pedestrian clip, the pipeline processes 150 frames, finds up to 9 people in a frame and tracks 18 people (indicative) plus 2 vehicles. The report is explicit about what was and was not measured: the Pi numbers are reference values, the development-host numbers are labelled as such, and accuracy on the organiser clip is not claimed without ground truth.

1. Societal context and application

1.1 The problem

Road traffic injuries are the leading cause of death for children and young adults aged 5–29 worldwide; more than half of all road deaths are among pedestrians, cyclists and motorcyclists, and 92% occur in low- and middle-income countries (WHO, 2026). India reported **4,61,312 crashes and 1,68,491 deaths in 2022**; two-wheeler riders were **44.5%** of the fatalities and pedestrians **19.5%** (MoRTH, 2023).

The data needed to act on a specific street is not complex. It is a count: *how many people cross here, when, and alongside how much traffic?* Yet that data is typically gathered by expensive cameras, proprietary traffic-analytics platforms or continuous cloud connectivity. Schools, ward offices and resident welfare associations rarely have any of those.

1.2 Why detection and counting help

A count converts a vague complaint ("traffic is bad at the school gate") into a comparable observation ("at 3:30 pm, 40 children cross with 60 vehicles in 15 seconds"). That is enough to prioritise a low-cost intervention: a crossing guard at dismissal, a zebra crossing, a speed hump, a no-parking zone within 20 metres of the gate, or a footpath repair. Counting is also repeatable: the same clip protocol before and after an intervention gives a crude but honest before/after measure.

Counting is deliberately less invasive than recognition. The system does not need faces, plates or persistent identities to answer the question, so it can be designed to discard them from the start.

1.3 Practical application: the school-zone safety audit

Who benefits: a school management committee, a ward road-safety officer or a resident welfare association.

Protocol:

1. Record three 15-second clips at the same gate: morning drop-off, afternoon dismissal and a quiet hour. Use a phone, a fixed viewpoint and the same direction.
2. Run each clip on the Pi with `namma-drishti run`. No internet is required.
3. Compare:
   - peak concurrent pedestrians and vehicles,
   - mean counts per frame,
   - observed person-seconds and vehicle-seconds,
   - the annotated frames, to sanity-check what the model counted.
4. If dismissal shows a high pedestrian count with a high vehicle count, use the table and two annotated frames as evidence for a crossing guard or a zebra crossing. Repeat after the intervention.

This is a decision-support workflow, not an enforcement system. The output is a count table and a handful of annotated frames, not a live surveillance feed.

2. Scope and requirements

2.1 Mandatory deliverables

Requirement	Implementation
Process an organiser 10–20 s clip or 10–20 images	`data/samples/pedestrians_15s.mp4` (150 frames); CLI also accepts image folders
Detect at least one category and display counts per frame	Six categories; `counts.csv`, `summary.txt`, HUD on annotated frames
Submit category + counts + at least two sample outputs	Contact sheet, two annotated Bengaluru images, annotated video
Report average inference time, model and throughput	`summary.json` / `benchmark_dev.json`; p50/p95, model-only and pipeline FPS
Explain societal issue, help and one application (≤2 min)	`docs/pitch-2min.md`; §1 and §7 here

2.2 Out of scope

Live camera, real-time surveillance, multi-object tracking as a product, dashboards, cloud connectivity and mobile applications are explicitly not required. The implementation avoids them so effort goes into the detector, measurement quality and evidence.

3. Model selection

3.1 Candidates

Arm publishes several ExecuTorch object detectors with Raspberry Pi 5 measurements. The relevant comparison (all on Pi 5, from Arm model cards) is:

Model	Format	Size	mAP@0.5:0.95	Pi 5 latency	Throughput
YOLO11n INT8	ExecuTorch, XNNPACK	**4.73 MB**	**38.90%**	**134.10 ms p50**	**7.46 FPS**
YOLOv5s INT8	ExecuTorch, XNNPACK	7.13 MB	36.09%	258.07 ms p50	3.87 FPS
YOLOv8s INT8	ExecuTorch, XNNPACK	13.24 MB	43.86%	457.03 ms p50	2.19 FPS
YOLOX-s INT8	ExecuTorch, XNNPACK	14.28 MB	40.01%	578.44 ms p50	1.73 FPS
YOLO12l INT8	ExecuTorch, XNNPACK	33.81 MB	52.29%	2650.58 ms p50	0.38 FPS
YOLO26n FP32	Ultralytics ExecuTorch	9.4 MB	~48.0%	~142 ms	~7.0 FPS

The YOLO26n FP32 export has a higher mAP but is roughly twice the size and has no INT8 optimization in the current Ultralytics export path. For a one-day hackathon on CPU, YOLO11n INT8 is the best balance of size, speed and accuracy, and its model card is verified on the exact target.

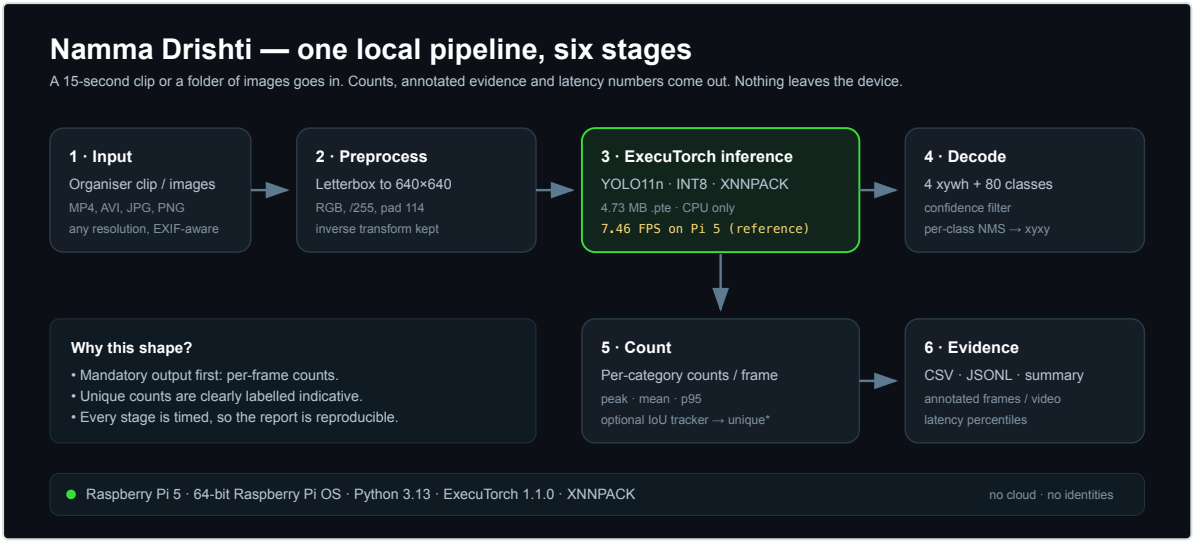
3.2 Why this artifact

- **Small:** 4.73 MB, 59.17 MB peak memory on Pi 5.
- **Fast enough:** 7.46 FPS; a 150-frame clip takes about 20 seconds of model time.
- **Accurate enough for counting:** 38.90% mAP@0.5:0.95, only 0.40 pp below the FP32 baseline.
- **Verified on the target:** Arm publishes p50/p90/p99, load time, memory and delegation for the Pi 5.
- **Reproducible:** the `.pte` is committed with a SHA-256, and `scripts/download_model.sh` re-fetches the same artifact.

3.3 Accuracy caveats

The reported accuracy is COCO val2017. It is not accuracy on Indian streets, and it is not accuracy on the organiser clip. COCO contains 80 classes; auto-rickshaws, hand carts and several local vehicle types are not among them. Dense, occluded scenes and small distant objects are the model's weak point. These are discussed in §8 rather than hidden.

4. System architecture



Pipeline

4.1 Preprocessing

The model's export contract is fixed: `[1, 3, 640, 640]`, RGB, `[0, 1]`, no mean/std normalization, letterbox pad 114. The pipeline:

- reads video frames with OpenCV (BGR) and phone images with PIL, honouring EXIF orientation;
- resizes the uint8 pixels with `cv2.INTER_LINEAR` to preserve aspect ratio;
- pads to 640×640 with a centre-anchored 114-grey canvas;
- converts BGR→RGB, HWC→CHW, uint8→float32 and divides by 255;
- keeps the scale and pad offsets for the inverse box transform.

The order matters: resizing after float conversion, or using PIL's antialiased bilinear downscale, shifts boxes and scores relative to the model card's protocol. The implementation is unit-tested against known transforms.

4.2 Inference

`ExecuTorchDetector` loads the `.pte` once via `Runtime.get()` and `program.load_method("forward")`. Each frame is a contiguous float32 tensor of shape `[1, 3, 640, 640]`. The model-only timer wraps `Method.execute`. The runtime is pinned to ExecuTorch 1.1.0 on the Pi; the development host uses 1.5.1, and the same 1.1.0 `.pte` loads in both.

4.3 Decode and NMS

YOLO11 output is `[1, 84, 8400]`: four xywh centre coordinates plus 80 class scores, with no separate objectness. The decoder:

1. normalises `[1, 84, N]`, `[84, N]`, `[N, 84]` and YOLOv5-style `[N, 85]` shapes to `[N, C]`;
2. computes the best score among the requested classes (or objectness × class for YOLOv5-style outputs);
3. filters below the confidence threshold;
4. converts xywh → xyxy;

5. applies greedy per-class NMS in NumPy (IoU 0.7, top 30,000 candidates, top 300 detections);
6. inverts the letterbox transform and clamps to the original image;
7. drops boxes thinner than 2 px.

Per-class NMS keeps a person and a car that overlap, while suppressing duplicate boxes of the same class. The decoder is dependency-light and unit-tested with synthetic outputs.

4.4 Counting and optional tracking

Mandatory counts are computed directly from detections per frame, so they do not depend on tracking quality. An optional tracker adds indicative unique counts:

- greedy one-to-one matching per class by IoU (0.25) on a constant-velocity predicted box;
- a centre-distance fallback for fast motion, restricted by class and size ratio;
- confirmation after 2 hits, coasting for up to 5 missed frames.

The tracker is not a production MOT system; IDs can switch when people cross or leave frame. Every output labels the unique numbers as indicative.

4.5 Evidence bundle

Each run writes:

File	Contents
`summary.txt`	Human-readable tables for counts, latency, exposure, reference
`summary.json`	Machine-readable metrics, model metadata, environment, outputs
`counts.csv`	Frame index, source frame, timestamp, per-class counts, stage timings
`detections.jsonl`	Per-frame boxes, scores, classes and optional track IDs
`REPORT.md`	Self-contained Markdown report generated from the summary
`annotated/`	Full-resolution annotated frames
`contact_sheet.jpg`	3–6 frame overview for quick review
`annotated.mp4`	Optional annotated clip

5. Implementation

5.1 Repository

```
src/namma_drishti/
  constants.py      COCO labels, default classes, model metadata
  preprocess.py     letterbox, EXIF-aware image read, inverse transform
  postprocess.py    decode, NumPy NMS, per-class counts
  tracker.py        IoU/velocity tracker for indicative unique counts
  model.py          ExecuTorch loading, timing, model info
  pipeline.py       video/image loop, evidence writing, benchmark
  report.py         terminal summary and Markdown run report
  visualize.py      annotated frames, contact sheet, title cards
  cli.py            run / benchmark / info / doctor
```

Tests cover letterbox geometry and padding, inverse mapping, decode shapes, YOLOv5 objectness, NMS suppression, class filtering, rescaling, tracker identity, confirmation, class separation, latency percentiles, stride-adjusted person-seconds and overlap shares. `make test` runs 21 tests.

5.2 Raspberry Pi setup

```
bash scripts/setup_pi.sh
```

The script:

- checks aarch64/arm64 and Python ≥ 3.10;
- creates `.venv`;
- installs pinned `torch==2.10.0`, NumPy, OpenCV and the ExecuTorch runtime support set from the CPU index;
- installs `executorch==1.1.0` with `--no-deps` (avoiding the x86-only `coremltools` dependency) and then installs the package editable;
- downloads the model if absent and verifies the SHA-256;
- runs `namma-drishti doctor`.

The prebuilt model is committed, so the core demo does not need a download.

5.3 CLI

```
namma-drishti run -i clip.mp4 -o results/clip --save-video
namma-drishti run -i images/ -o results/images --annotate-every 1
namma-drishti benchmark --input street.jpg --runs 50 --warmup 10
namma-drishti info --json
namma-drishti doctor
```

`run` accepts `--classes`, `--conf`, `--iou`, `--stride`, `--max-frames`, `--annotate-every`, `--no-track`, `--threads` and `--quiet`. The model path is configurable, so an organiser-approved YOLO-style `.pte` can be substituted without code changes.

6. Evaluation methodology

6.1 Data

- **Primary sample:** `pedestrians_15s.mp4`, a 15-second trim (150 frames, 640×480, 10 FPS) of the public OpenCV `vtest.avi` pedestrian sequence.
- **Stress test:** `bengaluru_traffic.jpg`, a dense elevated Bengaluru traffic scene (Wikimedia Commons, CC0).
- **Close-range test:** `bengaluru_street.jpg`, a street scene with a car, auto-rickshaw, motorcycle and pedestrians (Wikimedia Commons, public domain).

The organiser clip was not available at development time. The pipeline is input-agnostic and will process it at the finals.

6.2 Latency protocol

- **Warmup:** 3 inferences in the pipeline; 10 in the standalone benchmark.
- **Measured runs:** all 150 pipeline frames; 50 standalone inferences on a real image.
- **Stages:** read, preprocess, inference, decode/NMS, tracking, annotation.
- **Statistics:** mean, p50, p95, p99, min, max, standard deviation.
- **Throughput:** model-only `1000 / mean(inference_ms)`; pipeline `1000 / mean(total_ms)`. They are reported separately.
- **Memory:** `ru_maxrss` for the whole process; Arm's model-only peak memory is reported separately.
- **Device:** Apple Silicon macOS for the committed runs; Raspberry Pi 5 reference from Arm's model card. The two are never combined into one claim.
- **Video encode:** annotated-video encoding is an output convenience, excluded from the pipeline latency. Model-only and per-stage timings are unaffected.

6.3 Counts protocol

- Categories: person, bicycle, car, motorcycle, bus, truck.
- Confidence threshold: 0.35 for the committed pedestrian run; 0.35 for images.
- IoU threshold: 0.7.
- Per-frame counts are the mandatory metric.
- Unique counts are indicative, from the tracker.
- No ground-truth annotations were created for the sample media, so the report does not claim precision/recall or mAP on these clips.

6.4 What was not measured

- Accuracy on the organiser clip (no ground truth).
- Accuracy on Indian streets (COCO reference only).
- Pi 4 throughput (the pipeline runs, but no measurement is claimed).
- Low-light, rain, night or motion-blur performance.
- Long-duration drift, thermal throttling or power consumption.

These are stated explicitly so the results are not over-read.

7. Results

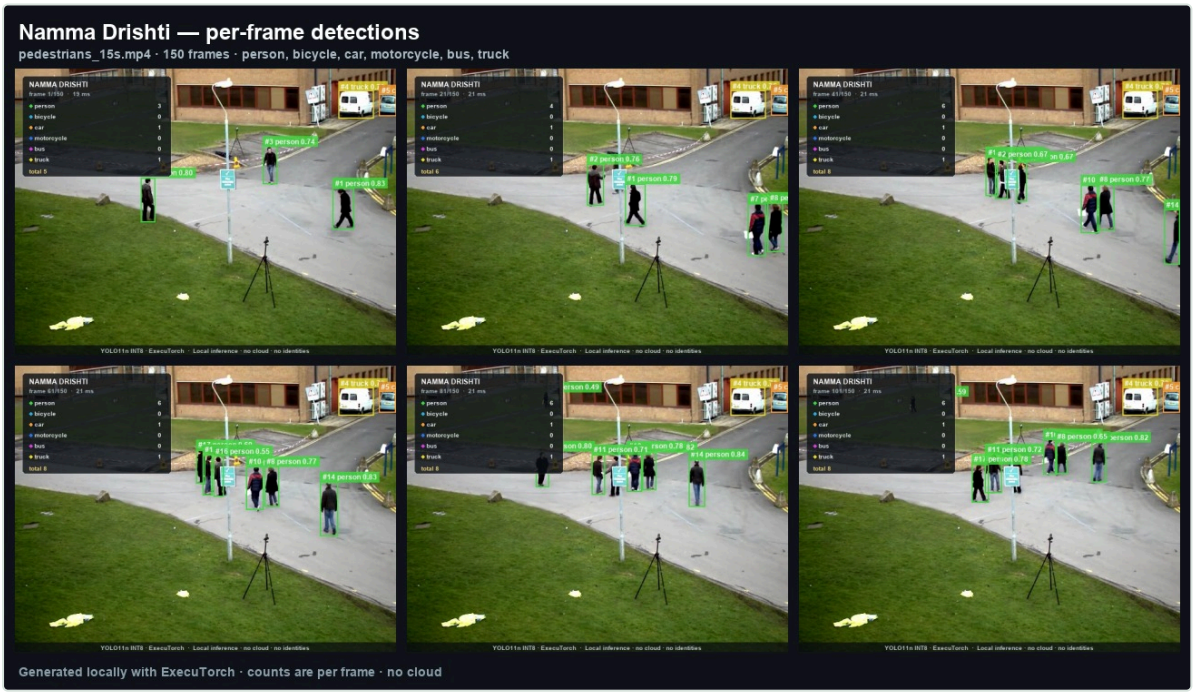
7.1 Pedestrian clip

Command:

```
namma-drishti run -i data/samples/pedestrians_15s.mp4 \
  -o results/demo_pedestrians --save-video --annotate-every 10
```

Category	Peak/frame	Mean/frame	p95	Unique (indicative)
person	9	5.50	8	18
bicycle	0	0.00	0	0
car	1	1.00	1	1
motorcycle	0	0.00	0	0
bus	0	0.00	0	0
truck	1	1.00	1	1

The car and truck are background vehicles — a parked white van and a blue car; both are real detections, but they illustrate that per-frame counts include stationary and background vehicles. This matters for a traffic-exposure metric and is discussed in §8.



Contact sheet

The tracker's 18 unique people over 15 seconds is plausible for this sequence, but it is indicative. Per-frame counts (peak 9, mean 5.5) are the defensible numbers.

7.2 Dense Bengaluru traffic stress test



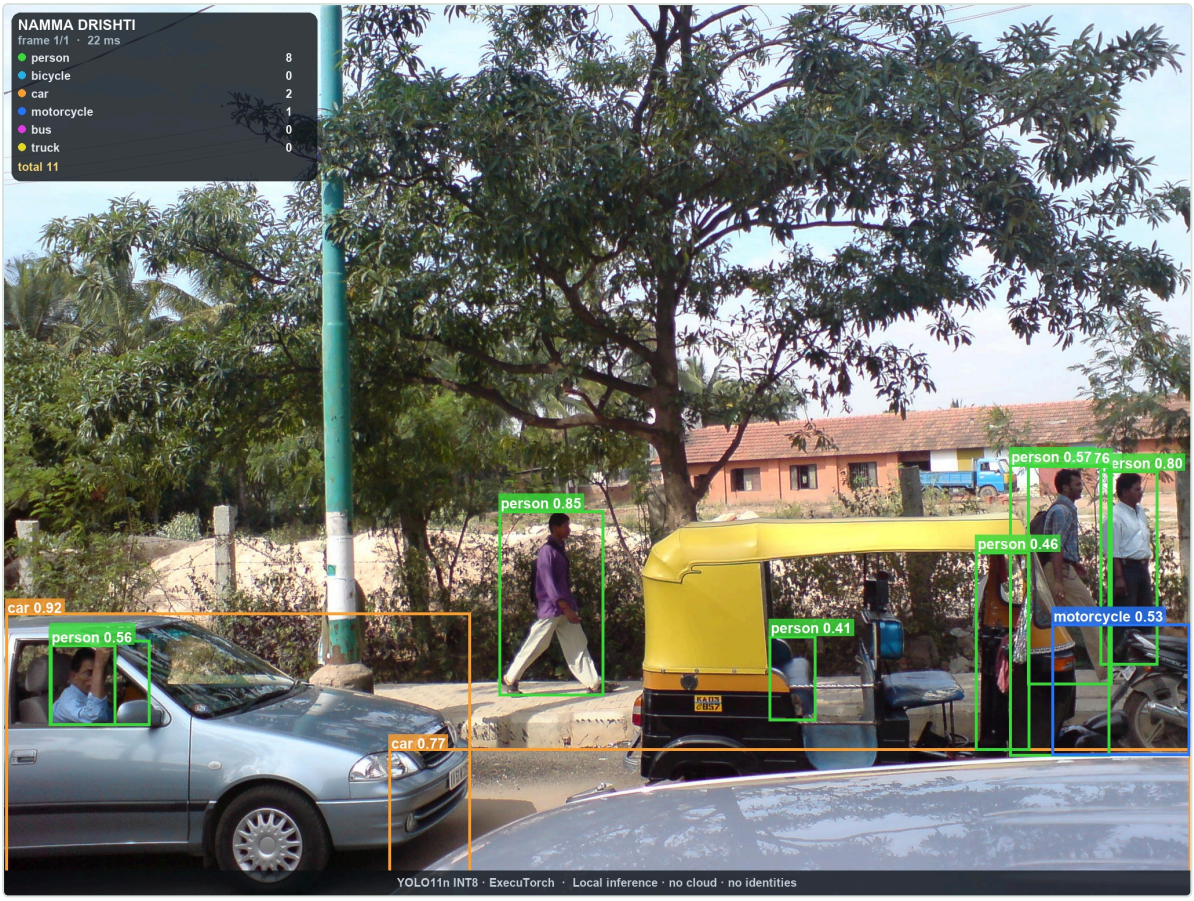
Dense traffic

Detected: **17 cars, 1 bus, 1 truck, 2 people** in one frame. The scene contains many motorcycles and auto-rickshaws that are not detected or not representable:

- **motorcycle: 0** despite clearly visible riders — small, occluded objects at 640×640 are the model's weak point;
- auto-rickshaws are not a COCO class, so they are missed or mapped to the nearest class;
- distant vehicles in the opposite lane are mostly below the confidence threshold.

This is a stress test, not an accuracy benchmark. It demonstrates the honest limit of a COCO model on a dense Indian road and motivates fine-tuning/tiling as future work.

7.3 Close-range Bengaluru street test



Street scene

Detected: **8 people, 2 cars, 1 motorcycle**. The large, close objects are detected confidently (0.77–0.92 for cars, up to 0.85 for people). The auto-rickshaw in the centre is not detected, because it is not a COCO class. This is the clearest illustration of the India-specific class gap.

7.4 Development-host latency

Environment: Apple Silicon macOS, ExecuTorch 1.5.1, torch 2.15.0, CPU, default threads. **This is not a Raspberry Pi measurement.**

Pipeline run (150 frames):

Stage	Mean ms	p50 ms	p95 ms	FPS from mean
Model inference	20.71	20.68	21.36	48.29
Preprocess	0.59	0.57	0.65	1697.79
Decode + NMS	0.67	0.65	0.71	1484.61
Frame read	0.19	0.16	0.22	5145.61
Annotate/save	2.92	2.81	3.72	342.66
Pipeline total	**25.39**	**25.20**	**26.29**	**39.38**

Standalone benchmark (50 runs, 10 warmups, real traffic image):

Metric	Value
Mean	20.34 ms
p50	20.16 ms
p95	20.81 ms
Throughput	49.16 FPS
Peak process RSS	437.8 MB (includes Python/PyTorch)

The standalone and pipeline inference numbers differ slightly because of frame content, thermal state and measurement overhead; both are reported rather than cherry-picked.

7.5 Raspberry Pi 5 reference

From Arm's model card (ExecuTorch 1.1.0, XNNPACK + KleidiAI, 1 thread, Pi 5 Cortex-A76):

Metric	Value
p50 latency	134.10 ms
p90 latency	134.68 ms
p99 latency	135.06 ms
Throughput	7.46 FPS
Model load	8.82 ms
Model size	4.73 MB
Peak memory	59.17 MB
mAP@0.5:0.95	38.90%
mAP@0.5	54.66%

At 7.46 FPS, 150 frames take about **20.1 seconds** of model time. A 20-second, 30 FPS organiser clip (600 frames) would take about **80 seconds** of model time. These are projections from the reference latency, not measured clip runtimes.

7.6 Comparison summary

Claim	Value	Basis
Model fits on edge	4.73 MB	Arm model card
Pi 5 speed	134.10 ms p50 / 7.46 FPS	Arm model card
Pi 5 memory	59.17 MB peak	Arm model card
COCO accuracy	38.90% mAP@0.5:0.95	Arm model card
Committed clip counts	150 frames, peak 9 people, 18 tracked people	This repo
Dev-host model latency	20.34 ms mean / 49.16 FPS (standalone); 20.71 ms in the 150-frame run	This repo, labelled dev
Tests	21 passing	This repo

8. Discussion

8.1 The India-specific class gap

COCO's 80 classes do not include auto-rickshaws, cycle-rickshaws, hand carts or several other vehicles that dominate Indian streets. A COCO detector can still count people, cars, motorcycles, buses and trucks, but it cannot give a complete vehicle census. For the proposed school-zone audit, that is acceptable: the decision is about pedestrian exposure and motorised traffic, and the six categories cover most of it. For a production deployment, fine-tuning on an Indian-road dataset is the right next step.

8.2 Small and occluded objects

The dense traffic image shows the failure clearly: 0 motorcycles detected despite many visible riders. At 640×640, a rider 40 pixels tall is a handful of pixels after downscaling. Options:

- **Tiled inference:** split the frame into overlapping tiles, run the model on each, merge detections. Cost: 4–9× inference time.
- **Higher input resolution:** re-export at 960 or 1280; cost: more compute and memory.
- **Fine-tuning:** adapt to the local domain; requires labelled data.
- **Zone-of-interest cropping:** for a fixed school-gate camera, crop the crossing area and run at native resolution; cost: one configuration per site.

The current PoC chooses the simplest, fastest configuration and documents the limitation.

8.3 Stationary vehicles and exposure metrics

Per-frame counts include parked vehicles. The pedestrian clip's background van and car are counted in every frame, which inflates the vehicle-seconds and the "frames with both" share. For a school-zone audit, the useful vehicle metric is *moving* vehicles in the crossing zone. Future work: use tracker displacement to exclude stationary tracks, or define a polygon zone and count only tracks that enter it. The current report labels the exposure numbers as descriptive, not a conflict measure.

8.4 Tracker limitations

The IoU/velocity tracker is intentionally simple. It can switch IDs when two people cross, and it can create a new ID after a long occlusion. That is why unique counts are labelled indicative and the per-frame counts remain the mandatory output. A production system would use a motion model, appearance features or a dedicated MOT algorithm; none of that is required for this PoC.

8.5 Privacy and power

The pipeline makes no network calls in the inference path. The model and code are local, and the default output is aggregate counts plus a few annotated frames. The tool does not perform face recognition or licence plate recognition. It is still a camera-based system; deployments should use signage, minimise retention, and prefer aggregate outputs. Power consumption was not measured; a Pi 5 with a 5V/5A supply is the reference platform.

9. Responsible AI

- **Purpose limitation:** infrastructure planning and safety audits, not enforcement or individual tracking.
- **Data minimisation:** aggregate counts by default; no identities; no cloud.
- **Transparency:** every number is traceable to a CSV/JSON file and a command; the report separates measured from reference values.
- **Fairness and bias:** COCO is not representative of Indian streets; the report names the gap rather than claiming general accuracy.
- **Human oversight:** the output is decision support; a human interprets the counts and checks the annotated frames.
- **Licence compliance:** YOLO11n and the Arm `.pte` are AGPL-3.0; the repository is AGPL-3.0 with the model notice included.

10. Reproducibility

```
# Pi
bash scripts/setup_pi.sh
source .venv/bin/activate
namma-drishti doctor
namma-drishti run -i data/samples/pedestrians_15s.mp4 \
  -o results/pi_run --save-video
namma-drishti benchmark --input data/samples/bengaluru_traffic.jpg --runs 50

# Development host
pip install -r requirements-dev.txt
pip install -e .
make test
make demo
```

- Model SHA-256: `154c54b52389219ebd982e1003227c1abbc3f29a99bdad241147b684d3041b96`
- Tests: `make test` → 21 passing.
- Sample media: `data/README.md`.
- Model card and licence: `models/README.md`.

11. Future work

1. **Fine-tune on Indian roads** — add auto-rickshaw, cycle-rickshaw and local vehicle classes; validate on manually counted clips.
2. **Tiled or higher-resolution inference** — improve small-object recall for dense traffic.
3. **Zone-based counting** — define a crossing polygon and count only tracks that enter it; exclude parked vehicles.
4. **Motion-aware exposure** — use tracker displacement to separate moving from stationary vehicles.
5. **Ground-truth protocol** — manually annotate 20 clips and report precision/recall per category, not just COCO mAP.
6. **Long-duration study** — thermal, power and drift measurements over a school day.
7. **Packaging** — a prebuilt Pi OS image or a systemd service for field use.

12. Conclusion

Namma Drishti demonstrates that a 4.73 MB INT8 ExecuTorch model on a CPU-only Raspberry Pi 5 can count pedestrians and vehicles in a short street clip at 7.46 FPS, with no cloud and no identities. The PoC ships a complete evidence bundle, a one-command Pi setup, 21 tests and an honest account of its limits. Its practical application — a school-zone safety audit — turns the counts into a specific, low-cost decision. The next step is to run it on the organiser clip, replace the reference numbers with on-device measurements, and validate the counts against a manual ground truth.

References

1. World Health Organization. *Road traffic injuries* fact sheet, 20 July 2026. https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries
2. Ministry of Road Transport and Highways, Government of India. *Road Accidents in India 2022*, 2023. https://morth.gov.in/backend/documents/uploaded/1755600426_RA_2022_30_Oct.pdf
3. Arm. *YOLO11n INT8 ExecuTorch for Raspberry Pi 5* model card. https://huggingface.co/Arm/yolo11n-int8-xnnpack-executorch-raspberrypi5
4. Ultralytics. *YOLO26 ExecuTorch integration* documentation. https://docs.ultralytics.com/integrations/executorch
5. PyTorch. *ExecuTorch documentation*. https://docs.pytorch.org/executorch/
6. Arm. *Optimize and deploy AI inference with ExecuTorch on Arm* course. https://learn.arm.com/courses/executorch-on-arm/
7. Namma AI Hackathon, NASSCOM CoE IoT. https://www.coe-iot.com/namma-ai-hackathon/
8. OpenCV. `vtest.avi` sample data. https://github.com/opencv/opencv/blob/4.x/samples/data/vtest.avi
9. Wikimedia Commons. *TRAFFIC VIEW IN BANGALORE* (CC0) and *Traffic in Bangalore* (public domain). See `data/README.md` for attribution.

Appendix A — exact commands for the committed evidence

```
# Pedestrian clip (video, tracking, annotated video)
namma-drishti run -i data/samples/pedestrians_15s.mp4 \
  -o results/demo_pedestrians --save-video --annotate-every 10

# Dense traffic image
namma-drishti run -i data/samples/bengaluru_traffic.jpg \
  -o results/demo_traffic --annotate-every 1 --no-track

# Street image
namma-drishti run -i data/samples/bengaluru_street.jpg \
  -o results/demo_street --annotate-every 1 --no-track

# Model-only benchmark
namma-drishti benchmark --input data/samples/bengaluru_traffic.jpg \
  --runs 50 --warmup 10 --output results/benchmark_dev.json

# Demo video
python scripts/make_demo_video.py
```

Appendix B — environment of the committed runs

Field	Value
Device	Apple Silicon (arm64), macOS
Python	3.12
ExecuTorch	1.5.1
torch	2.15.0
OpenCV	5.0.0
Model	YOLO11n INT8, 4.73 MB, SHA-256 verified
Input	640×480, 10 FPS, 150 frames (pedestrian clip)

The Raspberry Pi deployment pins ExecuTorch 1.1.0 and torch 2.10.0. The prebuilt `.pte` loads in both runtimes in our tests; the 1.1.0 pin matches Arm's model card and the Pi setup script. The exact Pi stack was also smoke-tested on the development host: 3 frames processed end-to-end, same output shape and counts, with 21.67 ms mean model latency. That reduces the risk of discovering a runtime/model mismatch only at the event.